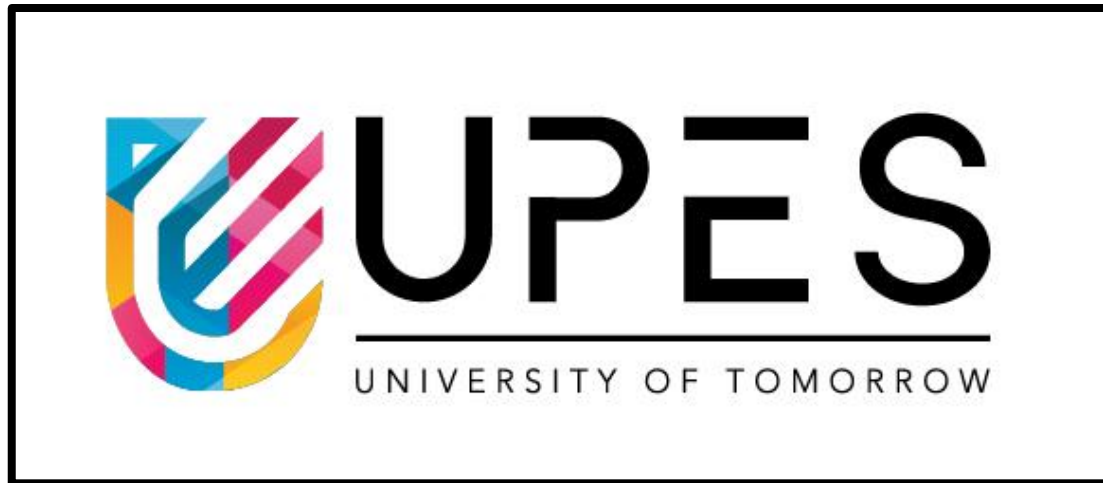




PROJECT:
NUMBER GUESSING GAME

NAME: NIKHIL SISODIA

SUBJECT: PROGRAMMING IN C

BATCH: B-28

SAP ID: 590025071

COURSE: B. TECH (CSE)

EMAIL ID: Nikhil.25071@stu.upes.ac.in

NUMBER GUESSING GAME

Abstract:

This project presents a simple, command-line implementation of a classic Number Guessing Game developed using the C programming language. The primary goal was to create a lightweight, engaging application that demonstrates fundamental procedural programming concepts and robust user interaction techniques.

Core Technology and Concepts

The application utilizes essential C standard libraries (`<stdlib.h>` and `<time.h>`) to implement the following core concepts:

1. Random Number Generation: The game uses the `srand()` and `rand()` functions to securely select a unique, unpredictable integer between 1 and 100.
2. Iterative Control Flow: A while loop manages the game's attempts, limiting the user to 10 guesses.
3. Conditional Logic: `if/else` statements are used to compare the user's input against the secret number, providing immediate feedback (hints like "Too High" or "Too Low").
4. Input Validation: Basic checks are included using `scanf` to prevent crashes when the user enters non-numeric characters.

Conclusion

The project successfully delivers a complete, executable game that highlights proficiency in C's control structures and library usage, providing a clear demonstration of random data generation and interactive console programming.

Part:1

```
C project_code.c X
Users > nikhilsisodia > Desktop > FOLDERS > code > C project_code.c > initialize_game(GameSettings *)
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #include <stdbool.h>
5
6  // --- Game Configuration Structure ---
7  typedef struct {
8      int max_number;      // The upper limit for the secret number
9      int max_guesses;     // Maximum attempts allowed
10     int secret_number;   // The number the user must guess
11     int attempts_taken;  // The actual number of guesses used
12     bool win_status;     // Did the player win?
13 } GameSettings;
14
15 // --- Function Prototypes ---
16 // Functions now accept a pointer to the GameSettings structure
17 void initialize_game(GameSettings *settings);
18 int get_user_guess(const GameSettings *settings); // const pointer: reads but doesn't change data
19 void check_guess(GameSettings *settings, int guess);
20 void print_game_info(const GameSettings *settings);
21 void game_over_message(const GameSettings *settings);
22
23 // -----
24
25 /**
26  * @brief Prints the structured information about the current game configuration.
27  */
28 void print_game_info(const GameSettings *settings) {
29     printf("-----\n");
30     printf("🎮 GAME CONFIGURATION\n");
31     printf("-----\n");
32     printf("Maximum Guessable Number: %d\n", settings->max_number);
33     printf("Total Attempts Allowed:  %d\n", settings->max_guesses);
34     printf("-----\n");
35 }
```

EXPLANATION:

The code sets up the foundation of a number-guessing game by including the necessary libraries, defining a structure to hold all game-related data, and writing functions that use this structure to manage the game's flow. The structure and functions work together so the entire game's state is stored in one place and passed easily between functions.

- The GameSettings struct stores values like the maximum number, allowed attempts, the secret number, attempts taken, and win status.
- Function prototypes declare the main game functions such as initialization, getting input, checking guesses, printing info, and showing the final result.
- print_game_info() prints out the game configuration using the data stored in the struct.

PART-2:

```
Users > nikhilsingh\Deskto\FOLDERS\code\C-project\code.c initialize_game(GameSettings *)
36 void initialize_game(GameSettings *settings)
37
38 --- Function Prototypes ---
39 /**
40  * @brief Prompts the user for a guess and handles input validation.
41  * @return The valid integer guess entered by the user.
42  */
43 void initialize_game(GameSettings *settings) {
44     // Set initial configuration and status
45     settings->max_number = 100;
46     settings->max_guesses = 10;
47     settings->attempts_taken = 0;
48     settings->win_status = false;
49
50     // Seed the random number generator
51     srand(time(NULL));
52
53     // Generate a random number between 1 and max_number
54     settings->secret_number = (rand() % settings->max_number) + 1;
55
56     // --- Welcome Messages ---
57     printf("=====\n");
58     printf("    Welcome to the C Number Guessing Game!    \n");
59     printf("=====\n");
60
61     print_game_info(settings);
62
63     printf("I have picked a secret number between 1 and %d.\n", settings->max_number);
64     printf("You have %d attempts to guess it.\n\n", settings->max_guesses);
65
66     /**
67     * @brief Prompts the user for a guess and handles input validation.
68     * @return The valid integer guess entered by the user.
69     */
70     int get_user_guess(const GameSettings *settings) {
71         int guess;
```

EXPLANATION:

This part of the code defines how the game is initialized and how the user's guess is taken. The initialization function sets all the starting values in the GameSettings struct, generates the random secret number, and prints the welcome message. The next function, get_user_guess, handles reading input from the user, validating it, and returning a clean, correct guess back to the main game loop.

- initialize_game() sets up the game by assigning default values, seeding the random number generator using time(NULL), generating a secret number between 1 and max_number, printing the welcome banner, and then showing the game configuration.
- get_user_guess() repeatedly asks the user for a number, checks if the input is valid, clears incorrect input from the buffer, ensures the guess is within range, and returns the valid guess.

PART-3:

```
71
72     while (1) {
73         printf("Attempt %d/%d. Enter your guess: ",
74             settings->attempts_taken + 1, // Accessing data via pointer
75             settings->max_guesses);
76
77         if (scanf("%d", &guess) != 1) {
78             printf("\n[ERROR] Invalid input. Please enter a whole number.\n");
79
80             // Clear the input buffer
81             int c;
82             while ((c = getchar()) != '\n' && c != EOF);
83             continue;
84         }
85
86         // Basic range check
87         if (guess < 1 || guess > settings->max_number) {
88             printf("\n[ERROR] Guess must be between 1 and %d.\n", settings->max_number);
89             continue;
90         }
91
92         return guess;
93     }
94 }
95
96 /**
97  * @brief Checks the user's guess against the secret number and updates game status.
98  */
99 void check_guess(GameSettings *settings, int guess) {
100     settings->attempts_taken++; // Update attempts via pointer
101
102     if (guess == settings->secret_number) { // Accessing data via pointer
103         settings->win_status = true;
104     } else if (guess < settings->secret_number) {
105         printf("--- Too Low! Try again. ---\n\n");
106     } else { // guess > secret_number
107         printf("--- Too High! Try again. ---\n\n");
108     }
109 }
```

EXPLANATION:

This section controls how each guess is evaluated and how the main game loop runs. The `check_guess` function increases the attempt counter, compares the user's guess with the secret number, and prints feedback telling the player whether their guess is too high, too low, or correct. The final part—inside `main()`—creates the game struct, initializes the game, repeatedly asks for guesses, checks them, and stops only when the player wins or uses all attempts.

- `check_guess()` increments the attempt count, checks the guess against the secret number, updates the win status, and prints the appropriate hint message.
- The while loop in `main()` runs the game: it keeps getting guesses and checking them until the player wins or reaches the maximum number of attempts.
- After the loop ends, `game_over_message()` prints whether the player won or lost and shows the secret number if necessary.

PART-4:

```
110
111  /**
112  * @brief Prints the final game outcome message.
113  */
114  void game_over_message(const GameSettings *settings) {
115      if (settings->win_status) {
116          printf("\n*** CONGRATULATIONS! ***\n");
117          printf("You guessed the number %d in %d attempts!\n",
118                settings->secret_number,
119                settings->attempts_taken);
120      } else {
121          printf("\n--- GAME OVER ---\n");
122          printf("You ran out of guesses. The secret number was %d.\n", settings->secret_number);
123      }
124  }
125
126  // -----
127
128  /**
129  * @brief Main function: controls the primary game loop.
130  */
131  int main() {
132      // 1. Declare the structure variable
133      GameSettings game_data;
134
135      // 2. Pass the ADDRESS (&) of the structure to initialize the game
136      initialize_game(&game_data);
137
138      // --- Main Game Loop ---
139      // Loop continues if the player hasn't won AND has attempts remaining
140      while (!game_data.win_status && game_data.attempts_taken < game_data.max_guesses) {
141
142          int current_guess = get_user_guess(&game_data);
143
144          // Pass the ADDRESS (&) of the structure to update its internal state
145          check_guess(&game_data, current_guess);
146      }
147
148      // Output final result
149      game_over_message(&game_data);
150
151      return 0;
152  }
```

EXPLANATION:

illustrates the clean and modular control flow of the C Number Guessing Game's final stages, leveraging a **GameSettings structure** and **pointers** to manage all data. The **game_over_message** function (lines 113-124) determines the final output by checking the boolean flag `settings->win_status` to decide between printing a victory message (including the `secret_number` and `attempts_taken`) or a loss message (revealing the `secret_number`). The **main** function (lines 131-152) acts as the program's driver, first declaring the master data variable `GameSettings game_data` and passing its **address** (`&game_data`) to all subsequent functions; its core responsibility is controlling the primary while loop, which continues only if the player has **not won** (`!game_data.win_status`) and **has attempts remaining**

(game_data.attempts_taken < game_data.max_guesses), ensuring the game terminates correctly before finally calling game_over_message(&game_data) to display the result based on the structure's final state.

- **Pointers for Efficiency:** Functions like game_over_message take a **pointer** (*settings) to the GameSettings structure, allowing them to access and read the current game state data efficiently using the **arrow operator (->)** without needing to copy the entire structure.
- **Decoupled Control:** The main function is decoupled from the implementation details; its main role is to manage the **flow** (initialization, looping, and final output), relying entirely on the dedicated functions and the shared data structure to handle the game's state and logic.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <stdbool.h>

// --- Game Configuration Structure ---
typedef struct {
    int max_number; // The upper limit for the secret number
    int max_guesses; // Maximum attempts allowed
    int secret_number; // The number the user must guess
    int attempts_taken; // The actual number of guesses used
    bool win_status; // Did the player win?
} GameSettings;

// --- Function Prototypes ---
// Functions now accept a pointer to the GameSettings structure
void initialize_game(GameSettings *settings);
int get_user_guess(const GameSettings *settings); // const pointer: reads but doesn't change data
void check_guess(GameSettings *settings, int guess);
void print_game_info(const GameSettings *settings);
void game_over_message(const GameSettings *settings);

// -----

/**
 * @brief Prints the structured information about the current game configuration.
 */
void print_game_info(const GameSettings *settings) {
    printf("-----\n");
    printf("🎮 GAME CONFIGURATION\n");
    printf("-----\n");
    printf("Maximum Guessable Number: %d\n", settings->max_number);
    printf("Total Attempts Allowed: %d\n", settings->max_guesses);
    printf("-----\n");
}
```

```

/**
 * @brief Sets up the game parameters and generates the secret number.
 */
void initialize_game(GameSettings *settings) {
    // Set initial configuration and status
    settings->max_number = 100;
    settings->max_guesses = 10;
    settings->attempts_taken = 0;
    settings->win_status = false;

    // Seed the random number generator
    srand(time(NULL));

    // Generate a random number between 1 and max_number
    settings->secret_number = (rand() % settings->max_number) + 1;

    // --- Welcome Messages ---
    printf("=====\n");
    printf(" Welcome to the C Number Guessing Game! \n");
    printf("=====\n");
    print_game_info(settings);
    printf("I have picked a secret number between 1 and %d.\n", settings->max_number);
    printf("You have %d attempts to guess it.\n\n", settings->max_guesses);
}

/**
 * @brief Prompts the user for a guess and handles input validation.
 * @return The valid integer guess entered by the user.
 */
int get_user_guess(const GameSettings *settings) {
    int guess;
    while (1) {
        printf("Attempt %d/%d. Enter your guess: ",
            settings->attempts_taken + 1, // Accessing data via pointer
            settings->max_guesses);
        if (scanf("%d", &guess) != 1) {
            printf("\n[ERROR] Invalid input. Please enter a whole number.\n");
            // Clear the input buffer
            int c;
            while ((c = getchar()) != '\n' && c != EOF);
            continue;
        }
        // Basic range check
        if (guess < 1 || guess > settings->max_number) {
            printf("\n[ERROR] Guess must be between 1 and %d.\n", settings->max_number);
            continue;
        }
        return guess;
    }
}

/**
 * @brief Checks the user's guess against the secret number and updates game status.
 */
void check_guess(GameSettings *settings, int guess) {

```

```

settings->attempts_taken++; // Update attempts via pointer
if (guess == settings->secret_number) { // Accessing data via pointer
settings->win_status = true;
} else if (guess < settings->secret_number) {
printf("--- Too Low! Try again. ---\n\n");
} else { // guess > secret_number
printf("--- Too High! Try again. ---\n\n");
}
}

/**
 * @brief Prints the final game outcome message.
 */
void game_over_message(const GameSettings *settings) {
if (settings->win_status) {
printf("\n*** CONGRATULATIONS! ***\n");
printf("You guessed the number %d in %d attempts!\n",
settings->secret_number,
settings->attempts_taken);
} else {
printf("\n--- GAME OVER ---\n");
printf("You ran out of guesses. The secret number was %d.\n", settings->secret_number);
}
}

// -----

/**
 * @brief Main function: controls the primary game loop.
 */
int main() {
// 1. Declare the structure variable
GameSettings game_data;

// 2. Pass the ADDRESS (&) of the structure to initialize the game
initialize_game(&game_data);

// --- Main Game Loop ---
// Loop continues if the player hasn't won AND has attempts remaining
while (!game_data.win_status && game_data.attempts_taken < game_data.max_guesses) {
int current_guess = get_user_guess(&game_data);
// Pass the ADDRESS (&) of the structure to update its internal state
check_guess(&game_data, current_guess);
}
// Output final result
game_over_message(&game_data);

return 0;
}

```

TEST CASES:

1.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
nikhilsisodia@Nikhils-MacBook-Air ~ % cd "/Users/nikhilsisodia/Desktop/FOLDER"

=====
GAME CONFIGURATION
=====
Maximum Guessable Number: 100
Total Attempts Allowed: 10
=====
I have picked a secret number between 1 and 100.
You have 10 attempts to guess it.

Attempt 1/10. Enter your guess: 63
--- Too High! Try again. ---

Attempt 2/10. Enter your guess: 17
--- Too High! Try again. ---

Attempt 3/10. Enter your guess: 73
--- Too High! Try again. ---

Attempt 4/10. Enter your guess: 74
--- Too High! Try again. ---

Attempt 5/10. Enter your guess: 82
--- Too High! Try again. ---

Attempt 6/10. Enter your guess: 64
--- Too High! Try again. ---

Attempt 7/10. Enter your guess: 55
--- Too High! Try again. ---

Attempt 8/10. Enter your guess: 68
--- Too High! Try again. ---

Attempt 9/10. Enter your guess: 82
--- Too High! Try again. ---

Attempt 10/10. Enter your guess: 64
--- Too High! Try again. ---

--- GAME OVER ---
You ran out of guesses. The secret number was 1.
nikhilsisodia@Nikhils-MacBook-Air code %
```

2.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
nikhilsisodia@Nikhils-MacBook-Air ~ % cd "/Users/nikhilsisodia/Desktop/FOLDER"

=====
Welcome to the C Number Guessing Game!
=====

=====
GAME CONFIGURATION
=====
Maximum Guessable Number: 100
Total Attempts Allowed: 10
=====
I have picked a secret number between 1 and 100.
You have 10 attempts to guess it.

Attempt 1/10. Enter your guess: 66
--- Too High! Try again. ---

Attempt 2/10. Enter your guess: 33
--- Too Low! Try again. ---

Attempt 3/10. Enter your guess: 55
--- Too High! Try again. ---

Attempt 4/10. Enter your guess: 44
--- Too High! Try again. ---

Attempt 5/10. Enter your guess: 48
--- Too High! Try again. ---

Attempt 6/10. Enter your guess: 35
--- Too Low! Try again. ---

Attempt 7/10. Enter your guess: 40
--- Too High! Try again. ---

Attempt 8/10. Enter your guess: 37
--- Too Low! Try again. ---

Attempt 9/10. Enter your guess: 36
--- Too Low! Try again. ---

Attempt 10/10. Enter your guess: 38

*** CONGRATULATIONS! ***
You guessed the number 38 in 10 attempts!
nikhilsisodia@Nikhils-MacBook-Air code %
```

3.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
cd "/Users/nikhilsisodia/Desktop/FOLDERS/code/" && gcc project_code.c -o proj
nikhilsisodia@Nikhils-MacBook-Air ~ % cd "/Users/nikhilsisodia/Desktop/FOLDERS/code/"project_code

=====
Welcome to the C Number Guessing Game!
=====

GAME CONFIGURATION
=====
Maximum Guessable Number: 100
Total Attempts Allowed: 10
=====
I have picked a secret number between 1 and 100.
You have 10 attempts to guess it.

Attempt 1/10. Enter your guess: 50
--- Too Low! Try again. ---

Attempt 2/10. Enter your guess: 7
--- Too Low! Try again. ---

Attempt 3/10. Enter your guess: 80
--- Too High! Try again. ---

Attempt 4/10. Enter your guess: 70
--- Too High! Try again. ---

Attempt 5/10. Enter your guess: 60
--- Too High! Try again. ---

Attempt 6/10. Enter your guess: 50
--- Too Low! Try again. ---

Attempt 7/10. Enter your guess: 55
--- Too Low! Try again. ---

Attempt 8/10. Enter your guess: 57

*** CONGRATULATIONS! ***
You guessed the number 57 in 8 attempts!
nikhilsisodia@Nikhils-MacBook-Air code %
```

4.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
cd "/Users/nikhilsisodia/Desktop/FOLDERS/code/" && gcc project_code.c -o proj
nikhilsisodia@Nikhils-MacBook-Air ~ % cd "/Users/nikhilsisodia/Desktop/FOLDERS/code/"project_code

=====
Welcome to the C Number Guessing Game!
=====

GAME CONFIGURATION
=====
Maximum Guessable Number: 100
Total Attempts Allowed: 10
=====
I have picked a secret number between 1 and 100.
You have 10 attempts to guess it.

Attempt 1/10. Enter your guess: 77
--- Too Low! Try again. ---

Attempt 2/10. Enter your guess: 88
--- Too Low! Try again. ---

Attempt 3/10. Enter your guess: 99
--- Too High! Try again. ---

Attempt 4/10. Enter your guess: 95
--- Too High! Try again. ---

Attempt 5/10. Enter your guess: 90
--- Too Low! Try again. ---

Attempt 6/10. Enter your guess: 93
--- Too High! Try again. ---

Attempt 7/10. Enter your guess: 92

*** CONGRATULATIONS! ***
You guessed the number 92 in 7 attempts!
nikhilsisodia@Nikhils-MacBook-Air code %
```